

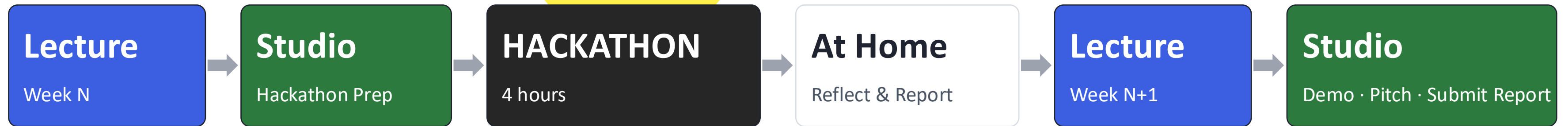
Hybrid AI Studio @ NLP Lab

Prof. Dr. Siegfried Handschuh
Götz-Henrik Wiegand & Lorena Raichle

Agenda

Week 2, 22. 09.2026

29.09.2026
14-18 Uhr
— Uhr



Hackathon 1

Derive from Knowledge
Knowledge

01 Team Names & Hackathon Planning

1

02 Syntax Exercises

2

03 Ontology Exercise

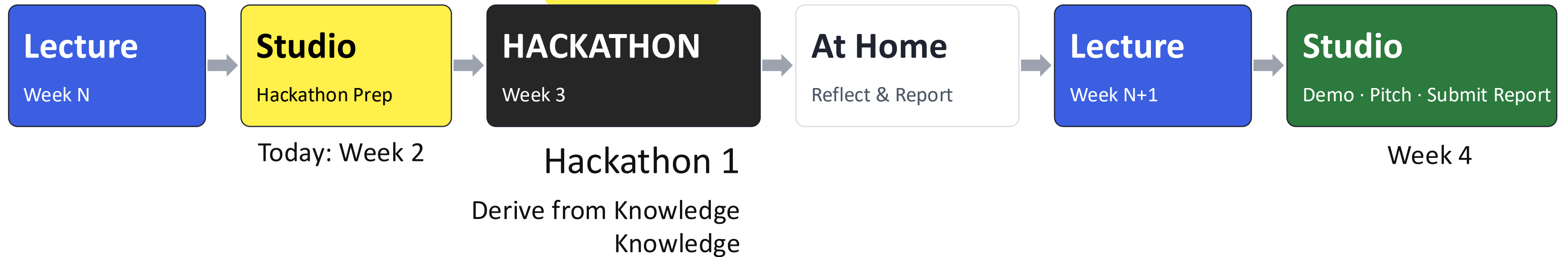
04 FrameX Python API

4

Agenda

Week 2, 22. 09.2026

29.09.2026
14-18 Uhr
— 6 Uhr



01 Team Names & Hackathon Planning

1

02 Syntax Exercises

2

03 Ontology Exercise

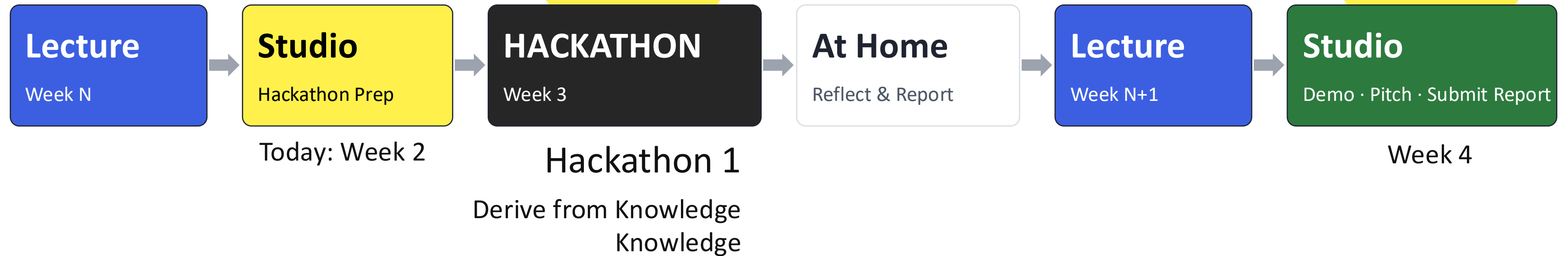
3

04 FrameX Python API

4

Agenda

Week 2, 22. 09.2026



01 Team Names & Hackathon Planning

1

02 Syntax Exercises

2

03 Ontology Exercise

3

04 FrameX Python API

4

Hackathon 1 Planning

Derive from Knowledge

What are your Hackathon Team Names?

▼ Hackathon Team 1

Full

3 / 3 students

⋮ Cyril Gabriele ⋮

⋮ Philipp Rechst... ⋮

⋮ Gian Seifert ⋮

▼ Hackathon Team 2

Full

3 / 3 students

⋮ Sascha Ciric ⋮

⋮ Silvan Fässler ⋮

⋮ Adriano Lama ⋮

▼ Hackathon Team 3

Full

4 / 4 students

⋮ Egor Levashe... ⋮

⋮ Ethan Jin Lim ⋮

⋮ Lucrezia Volp... ⋮

⋮ Seow Wee Sia... ⋮

Hackathon 1
Derive from Knowledge

29.09.2026
14-18 Uhr
— 18 Uhr

Hackathon 1: Derive from Knowledge

TASK

DERIVE FROM KNOWLEDGE

Turn a **real-world, messy open dataset** into a queryable **reasoning system**.

Not a queryable database, but a system that can derive new facts from rules and explain its own conclusions.

Team Roles

- 3 roles per Hackathon-Team during **flight mode**



DEVELOPER

Software & FrameX Architecture

- Owns the technical architecture of the system
- Works with the logic-reasoning framework and its integration
- Ensures the code runs, is structured, and can be extended



DOMAIN EXPERT

Dataset Understanding

- Builds a deep understanding of the data's structure, meaning, and limits
- Translates domain questions into concrete system requirements
- Spots data gaps and quality issues early



AUDITOR

Evaluation & Correctness

- Designs test questions and queries to probe the system
- Checks that derived facts and the underlying logic hold up
- Key question: how do we evaluate the system?

Rotation: Roles rotate after every hackathon. By the end, each person will have worked in each role once.

Requirements: what the system must deliver

DATA FOUNDATION

- 1 Heterogeneous real-world data ingestion

Pull real-world data (API access), tolerates missing fields, inconsistent types & encoding, ...

- 2 Design an ontology

Frames, class hierarchy, relations between typed objects, Rules and Facts

REASONING

- 3 Rule-based Derivation
Declarative logic, conclusions update automatically, nothing hardcoded

- 4 Explicit world assumption
Declare Open-world / Closed-world reasoning

- 5 Compositional, multi-hop querying
Joins and chained relations across derived and raw facts, not single-table lookups

- 6 Explainability & provenance
Every derived fact traces back to the rule and evidence that produced it

ENGINEERING QUALITY

- 7 Performance at scale
Hint: how can “fast reasoning” be ensured? Loading and inference cost?

- 8 Correctness & Reproducibility
Test queries are provided

- 9 Portability
Minimal dependencies: runs in constrained or restricted environments

Hackathon Deliverables

- Each Hackathon accounts for 30% of your final grade (3 Hackathons = 90%)

Inside the 4-Hour Hackathon

30 %

FlightMode · 0–1h

Offline. No phone, no laptop.
Concept & brainstorming.

Checkpoint · 1h

Concept handed in
on paper.

Open Work · 1–4h

Laptops allowed.
Build & iterate on the concept.

End of Hackathon · 4h

Code + Readme submission
via canvas.

40 %

First hour is fully offline (concept on paper only) -> remaining three hours are open build time, ending in a code submission.

After the hackathon:

teams reflect and write a short report at home before the next studio session, where they demo, pitch, and submit their final report.

30 %

Grading

3 Deliverables per Hackathon:

30/40/30

FlightMode Docs / Code / Demo & Report

Paper sheets
Ontology & Solution
Solution outline

FlightMode Docs

- No phones, laptops or other devices!
- Problem understanding, brainstorming and task discussion
- Working on first solution ideas on paper

→ Deliverable: handing in FlightMode Docs after FlightMode phase!

**GitHub
Template**

Code

- Code Submission latest at 18:15 after each Hackathon
- Code must be reproducible, readable and cleaned up
- Readme must contain all necessary instructions and project overview

→ Deliverable: zip file with Code & Readme via Canvas!

**Latex
Template**

Demo & Report

- Report is written between finished Hackathon and next Studio Session
- Report written in Latex with [IMRaD](#) structure (max. 8 pages content)
- Demo pitched with Slides in next Studio Session

→ Deliverable: Demo Slides & Report, both PDF!

What you will receive at Hackathon Start

- Task Sheet & Hackathon Introduction by TAs
- Challenge-Dataset Sneak Peek to get started and build first ontology
- Paper Sheets to get started for FlightMode Submission
- Report Latex Template

What you bring:

- Pen for FlightMode session
- Laptop prepared for Hackathon (Template, UV, Github etc.)



Coding Agents during the Hackathon

- Coding agents may be used after the Flight Mode phase. Not before!
- Declare tool + model used in your README
- Any tool is fine (Claude, Codex, ...)
- no-cost option: **OpenCode**, with free access to models

<https://opencode.ai/en>

<https://opencode.ai/docs>

<https://opencode.ai/docs/zen/#pricing>

MODEL	INPUT	OUTPUT	CACHED READ	CACHED WRITE
Big Pickle	Free	Free	Free	-
MiMo-V2.6-Flash Free	Free	Free	Free	-
MiMo-V2.5 Free	Free	Free	Free	-
Ling 3.0 Flash Fin Free	Free	Free	Free	-
Nemotron 3 Ultra Free	Free	Free	Free	-
Nemotron 3.5 Lightning Free	Free	Free	Free	-
Muse Spark 1.3 Contributor Free	Free	Free	Free	-
Jev 1.13 Free	Free	Free	-	-
Jev 1.13	\$0.042	Free	-	-
MiniMax M3	\$0.30	\$1.20	\$0.06	-
MiniMax M2.7	\$0.30	\$1.20	\$0.06	-
MiniMax M2.5	\$0.30	\$1.20	\$0.06	-
GLM 5.3 Flash	\$0.15	\$0.50	\$0.03	-

How to prepare for Hackathon 1

- **Technical Setup**

- Project Template Setup <https://github.com/unisg-ics-dsnlp/hybridai-h1-template>
 - Python 3.12, uv, and git installed
- Your development tools for Python and FrameX)
 - Code Editor, Coding Agent (e.g. OpenCode) etc.
- Familiar with FrameX Python API
<https://unisg-ics-dsnlp.github.io/FrameX-Doc/python/index.html>

- **Team**

- Roles assigned for first Hackathon: *Developer, Domain Expert, Auditor*

- **Theory**

- Familiar with FrameX and exercises: logic facts vs. rules vs. queries
- Understanding the Open / Closed World Assumption
- During **FlightMode**, you need to **sketch and submit a few example facts and rules**, and rough out how you'd structure your ontology (which classes, relations,...)
- Work through this week's Hackathon prep exercises
 - Syntax Exercises on Symbolic foundations: REPRESENT, DERIVE, MODEL, DECIDE.
 - Ontology Exercise



Template on GitHub

Syntax Exercises

Ontology Exercises

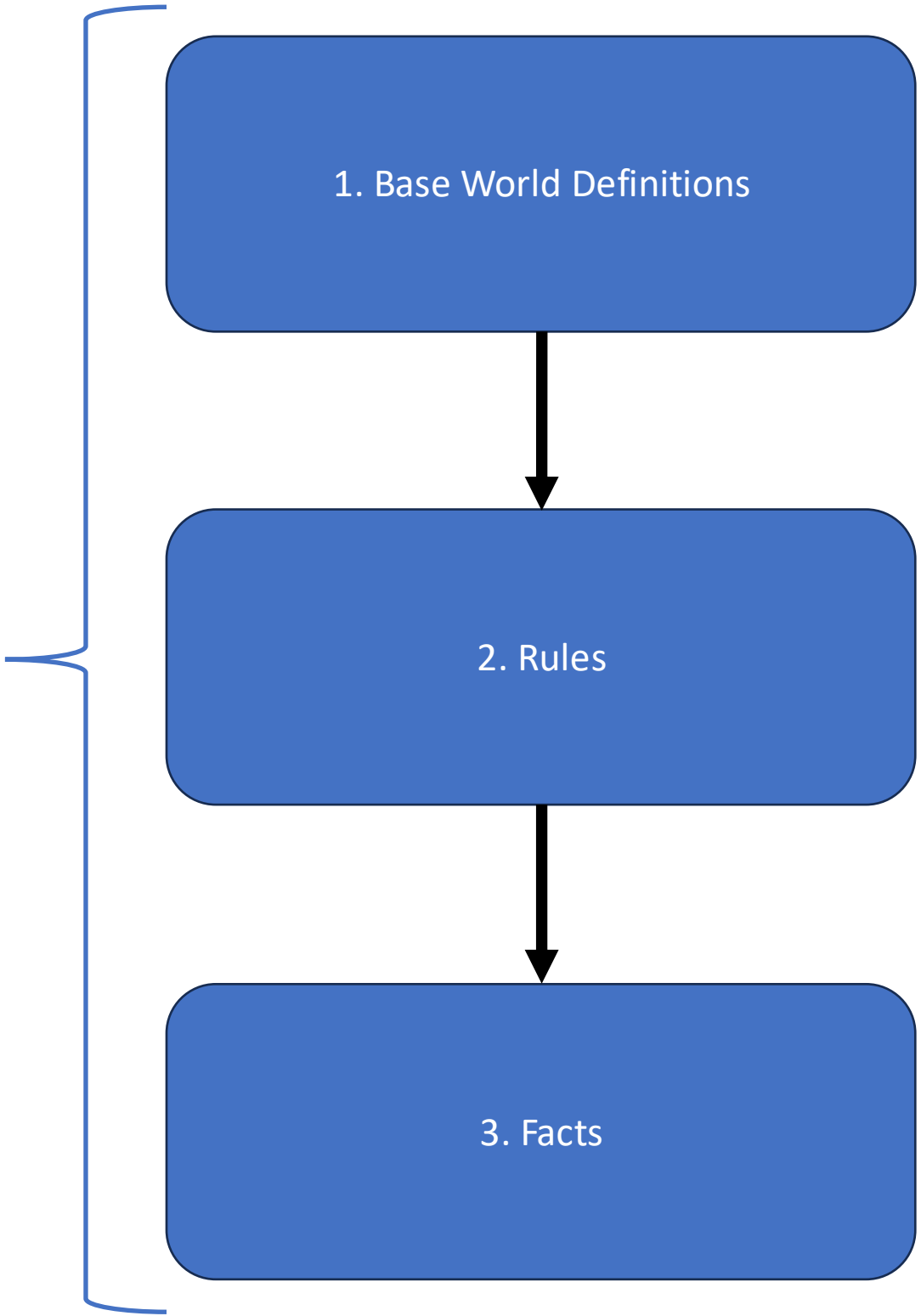
Derive From Knowledge: Titanic Ontology

- Titanic Dataset: Simple Dataset that displays the Titanic Passenger list and survival
- Goal: Create a logic rule system in FrameX that is derived from the Titanic Dataset
 - <https://www.kaggle.com/datasets/amineipad/titanic-dataset>

test.csv (28.63 kB)

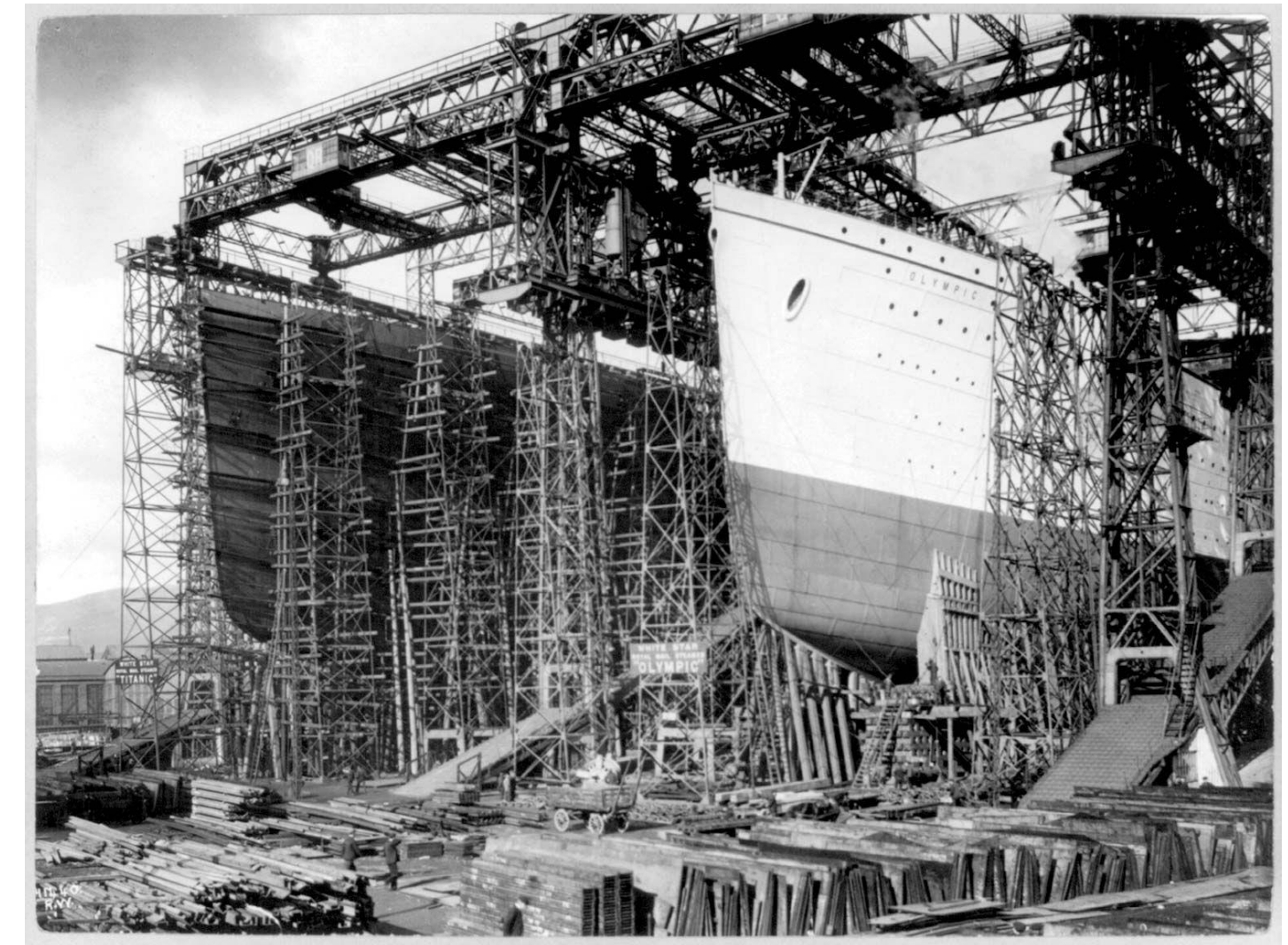
Detail Compact Column

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	
893	0	3	Wilkes, Mrs. James (Ellen Needs)	female	47	1	0	363272	7	
894	0	2	Myles, Mr. Thomas Francis	male	62	0	0	240276	9.6875	
895	0	3	Wirz, Mr. Albert	male	27	0	0	315154	8.6625	
896	0	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22	1	1	3101298	12.2875	
897	0	3	Svensson, Mr. Johan Cervin	male	14	0	0	7538	9.225	
898	0	3	Connolly, Miss. Kate	female	30	0	0	330972	7.6292	



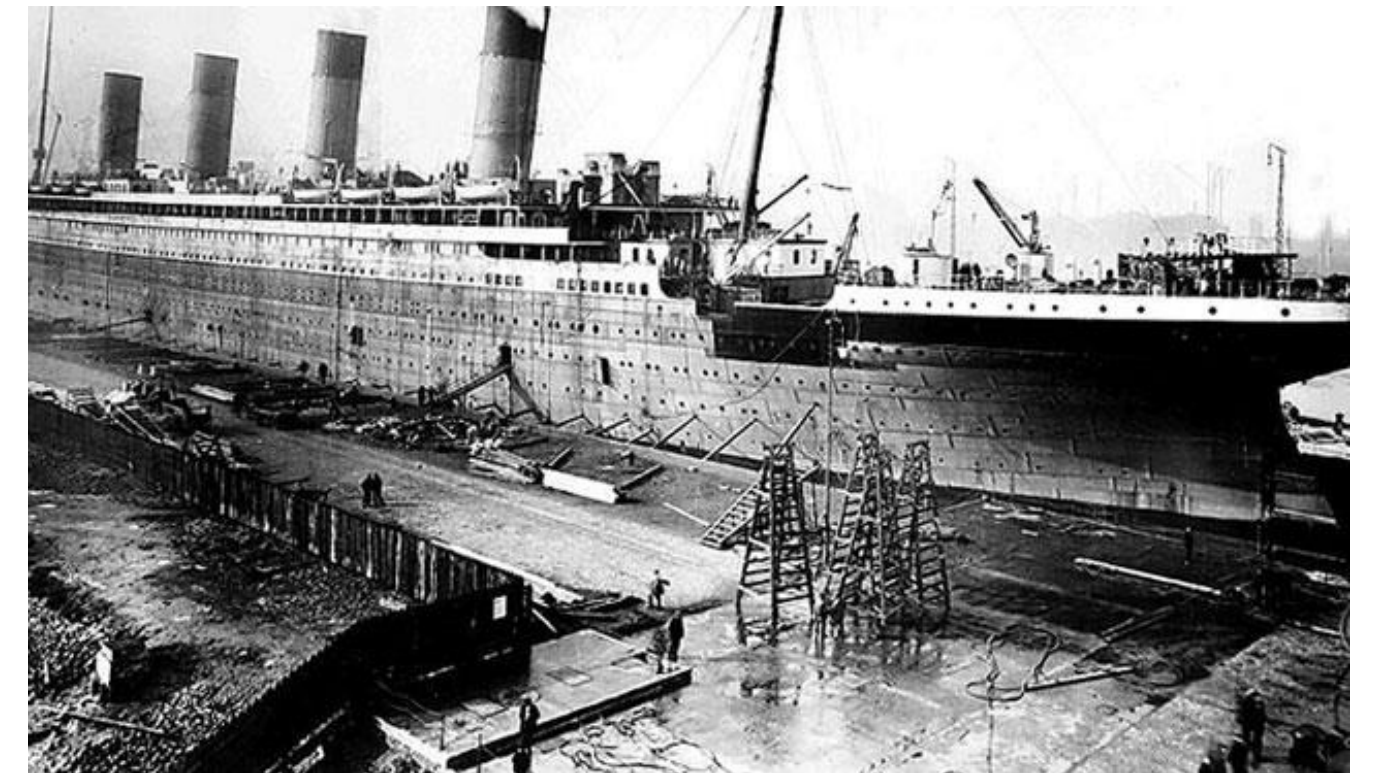
Step 1: Model the Base World

1. Start with World Assumption: Open World / Closed World?
2. Base Class: Model the Passenger
3. Derived Passenger Classes: Survivor, TravelingClass, Sex, Age, etc.
4. Model Property Characteristics: Define how many of e.G. names a person has
5. Required Property: Define what properties are required



Step 2: Define Rules

1. Survival rules: Encode “survived = 1” in to FrameX logic
2. Passenger Class Rules
3. Sex rules
4. Age rules
5. Family and traveling rules
6. Embarkation rules: Southampton/Cherbourg/Queenstown?
7. Cabin information
8. Passenger Relations: Same Ticket? How is that relation (asymmetric/symmetric)?
9. Higher Level Rules: Who is a Child-Survivor? Who is a First-Class Survivor etc.



Step 3: Facts

- After defining the World and the Rules we now need to fill it with facts
- Example Facts for Miss. Catherine “Katie” McCarthy:

```
passenger_1000:Passenger[  
  name -> "McCarthy, Miss. Catherine \"Katie\"";  
  survived -> 1;  
  pclass -> 3;  
  sex -> "female";  
  sibSp -> 0;  
  parch -> 0;  
  ticket -> "383123";  
  fare -> 7.75;  
  embarked -> "Q"  
].
```



FrameX Python API

FrameX-Python API

- To use FrameX outside the FrameX-Workbench there is a Python API
- Important for the first hackathon next week!
- The documentation of the Python API is here: <https://unisg-ics-dsnlp.github.io/FrameX-Doc/python/index.html>
- Self learning material for hackathon preparation is here: <https://unisg-ics-dsnlp.github.io/FrameX-Doc/python-exercise/index.html>

Python API

The FrameX Python Framework is a pure Python client for the FrameX reasoning engine, with zero third-party dependencies.

Where the [FrameX Syntax](#) describes *what* you write, the Python API describes *how* a program drives the engine: loading programs, asking queries and revising state.

```
from framex import Client

with Client() as client:
    client.load("world open.\n socrates:Human.\n ?X:Mortal <- ?X:Human.")
    result = client.query("?- socrates:Mortal.")
    print(result)  # {'status': 'true'}
```

In the following chapters, the Python API will be explained:

- [Installation and Quick Start](#)
- [Client](#)



Götz-Henrik Wiegand



Lorena Raichle

Questions?

Contact us via hybrid-ai@mailman.nlp-lab.ai

Hackathon 1

Derive from Knowledge
Knowledge

29.09.2026

14-18 Uhr

— 1hr